inner join

- only the matching records

t1 - 100 , c1, c2, c3

t2 - 100 , c4, c5, c6

50

Left outer join = 50 (matching records) + 50 non matching records from the left table

c1, c2, c3, c4, c5, c6 (50)
c1, c2, c3, null, null, null (50)

customers and orders

left outer join

```
SELECT o.order_id, date(o.order_date),o.order_customer_id,
o.order_status, c.customer_fname, c.customer_city,
c.customer_state
FROM customers c
LEFT JOIN orders o
ON o.order_customer_id = c.customer_id;

SELECT count(distinct customer_id)
FROM customers c
LEFT OUTER JOIN orders o
ON o.order_customer_id = c.customer_id;
```

-- customers who never placed any order

do a left outer join and filter out the null on the right side SELECT count(*)

```
FROM customers c
LEFT OUTER JOIN orders o
ON o.order_customer_id = c.customer_id
WHERE o.order_customer_id is null
```

when to use inner join vs left outer join?

right outer join
==========

```
SELECT o.order_id, date(o.order_date),
o.order_customer_id, o.order_status, c.customer_fname,
c.customer_city, c.customer_state
FROM customers c
LEFT JOIN orders o
ON o.order_customer_id = c.customer_id;
```

```
SELECT o.order_id, date(o.order_date),
o.order_customer_id, o.order_status,
c.customer_fname, c.customer_city, c.customer_state
FROM orders o
RIGHT JOIN customers c
ON o.order_customer_id = c.customer_id;
```

50 MATCHING RECORDS

15 NON MACHING RECORDS IN THE LEFT TABLE

23 NON MATCHING RECORDS IN THE RIGHT TABLE

FULL OUTER JOIN
==========

MATCHING RECORDS FROM BOTH TABLE

```
SELECT o.order_id, date(o.order_date),o.order_customer_id,
o.order_status, c.customer_fname, c.customer_city, c.customer_state
FROM customers c
LEFT JOIN orders o
ON o.order_customer_id = c.customer_id;
```

UNION

```
SELECT o.order_id, date(o.order_date),o.order_customer_id,
o.order_status, c.customer_fname, c.customer_city, c.customer_state
FROM customers c
RIGHT JOIN orders o
ON o.order_customer_id = c.customer_id;
```

WHEN YOU WILL USE FULL OUTER JOIN ?

from
join
where
group by
having
select
distinct
order by
limit
=========

cross join or cartesian join

100 * 200
====
20000

```
select count(*)
from customers c
join orders o;
```

-- its used very less frequently

can be used to generate all possible combinations of 2 datasets

```
Products
---------
| ProductID | ProductName |
|-----------|-------------|
| 1         | T-Shirt     |
| 2         | Hoodie      |
| 3         | Mug         |
---------------------------
Colors
--------
| ColorID | ColorName |
|---------|-----------|
| 1       | Red       |
| 2       | Blue      |
| 3       | Green     |
------------------------
SELECT p.ProductName, c.ColorName
FROM Products p
CROSS JOIN Colors c;

| ProductName | ColorName |
|-------------|-----------|
| T-Shirt     | Red       |
| T-Shirt     | Blue      |
| T-Shirt     | Green     |
| Hoodie      | Red       |
| Hoodie      | Blue      |
| Hoodie      | Green     |
| Mug         | Red       |
| Mug         | Blue      |
| Mug         | Green     |
---------------------------
```

cross join can be used to generate sample test data

customers
orders

top 3 states from where maximum number of orders are placed...

grouping column - states (customer)
orders is from orders table

```
SELECT o.order_id, date(o.order_date),o.order_customer_id, o.order_status,
c.customer_fname, c.customer_city, c.customer_state
FROM customers c
LEFT JOIN orders o
ON o.order_customer_id = c.customer_id;

SELECT c.customer_state, count(*) as num_orders
FROM customers c
INNER JOIN orders o
ON o.order_customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY num_orders desc
limit 3;

-- find top 5 customers with maximum orders in closed status

SELECT c.customer_id, count(*) as num_orders
FROM customers c
INNER JOIN orders o
ON o.order_customer_id = c.customer_id
where o.order_status = 'CLOSED'
GROUP BY c.customer_id
ORDER BY num_orders desc
limit 5;
```

CUSTOMER_ID (GROUPING COLUMN) - ORDERS

AGGREGATION (ORDER_ID) - ORDERS

FILTERING - ORDERS

```
SELECT order_customer_id, count(*) as num_orders
FROM orders
where order_status = 'CLOSED'
GROUP BY order_customer_id
ORDER BY num_orders desc
limit 5;
```

-- list the customer_street from where we have orders will all possible order status

(grouping) customer_street - customers

(aggregation) order_status - orders

```
SELECT c.customer_street as total
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.order_customer_id
GROUP BY c.customer_street
HAVING count(distinct o.order_status) = (select count(distinct order_status) from orders);
```